Validation

Validation · Signup form accumulates all field errors · Build with Waqar

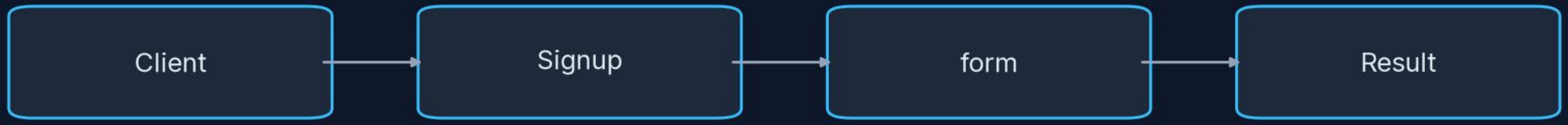
Today's Hook

- Problem: Signup form accumulates all field errors
- Pattern: Validation — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Signup form accumulates all field errors
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Validation: Signup form accumulates all field errors

Real-World Scenario

- Running example: Signup form accumulates all field errors
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.validation.problem;

import java.util.*;

public final class NaiveSignup {
  public static String signup(String email, int age){
    if(!email.contains("@")) throw new IllegalArgumentException("email");
    if(age<18) throw new IllegalArgumentException("age");
    return "ok:"+email;
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.validation.problem;

import java.util.*;
import java.util.function.*;

sealed interface Validation<E,A> permits Valid, Invalid {
  boolean isValid(); A get(); List<E> errors();
  default <B> Validation<E,B> map(Function<A,B> f){ return isValid()? new Valid<>(f.apply(get())): new Invalid<>(errors()); }
  default <B> Validation<E,B> zipWith(Validation<E,B> other, BiFunction<A,B,?> ignored){ throw new UnsupportedOperationException(); }
}
record Valid<E,A>(A value) implements Validation<E,A> {
  public boolean isValid(){return true;} public A get(){return value;} public List<E> errors(){return List.of();}
}
record Invalid<E,A>(List<E> errs) implements Validation<E,A> {
  public boolean isValid(){return false;} public A get(){throw new IllegalStateException();} public List<E> errors(){return errs;}
}
final class Signup {
  static Validation<String,String> email(String e){ return e!=null && e.contains("@")? new Valid<>(e): new Invalid<>(List.of("email has no @")); }
  static Validation<String,Integer> age(int a){ return a>=18? new Valid<>(a): new Invalid<>(List.of("age under 18")); }
  static Validation<String,String> form(String e, int a){
    var ve=email(e); var va=age(a);
    if (ve.isValid() && va.isValid()) return new Valid<>("welcome "+ve.get());
    List<String> errs=new ArrayList<>(); errs.addAll(ve.errors()); errs.addAll(va.errors());
    return new Invalid<>(errs);
  }
```

Solution (2)

```
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.validation.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Validation: signup accumulates errors ===");
    System.out.println("--- naive ---");
try { System.out.println(NaiveSignup.signup("bad", 10)); } catch(Exception e){ System.out.println("caught="+e.getMessage()); }
System.out.println("--- validation ---");
System.out.println(Signup.form("a@b.c", 20));
System.out.println(Signup.form("bad", 10));
  }
}
```

Walkthrough Output

```
=== Validation: signup accumulates errors ===
--- naive ---
caught=email
--- validation ---
Valid[value=welcome a@b.c]
Invalid[errs=[email has no @, age under 18]]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Validation.

Related Patterns

- Contrast Validation with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Validation → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.